

Bangladesh University of Engineering and Technology

Department of Computer Science and Engineering

CSE 406: Computer Security Sessional

TCP SYN Flood + DoS Attack and Defense Mechanisms

Submitted By:

Shahriar Ahmed Seam

Student ID: 2005093

Hozifa Rahman Hamim

Student ID: 1705083

September 12, 2026

Abstract

TCP (RFC 793 [1]) sets up connections through a three-way handshake: the client sends a SYN, the server replies with a SYN-ACK and allocates memory for a Transmission Control Block (TCB), and the client finishes with an ACK. The trouble is that this process is lopsided—sending a 40-byte SYN costs the client almost nothing, but the server has to commit real kernel memory (tracked as `request_sock` in Linux [6]) and hold a slot in its backlog queue before it even knows whether the client is legitimate.

For this project, we wrote a TCP SYN flood attack tool from scratch in C using POSIX raw sockets (`SOCK_RAW` with `IP_HDRINCL`). We did not use any third-party tools or packet-crafting libraries, as per course policy. Instead, we built the IPv4 and TCP headers byte by byte, implemented the RFC 1071 [3] checksum with TCP pseudo-headers ourselves, and ended up with an injection loop that pushes over 388,000 packets per second.

We tested this inside an isolated virtual network. With defenses turned off (`tcp_syncookies=0`), the attack filled the server's backlog queue in roughly 25.6 ms, and legitimate connections failed 100% of the time. We then implemented and evaluated four defenses: (1) SYN Cookies (`tcp_syncookies=1`), which let the server avoid allocating any state for incoming SYNs and brought legitimate success rates back to 90–100%; (2) `iptables` rate limiting, which dropped 84.7% of attack packets before they reached the socket layer; (3) Netfilter `SYNPROXY`; and (4) kernel parameter hardening via `sysctl`. We verified all of these through terminal traces, socket queue telemetry (`ss`, `netstat`), and client latency measurements.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Problem Statement	1
1.3	Project Objectives and Scope	1
1.4	Report Organization	2
2	Background & TCP Protocol Mechanics	3
2.1	The TCP State Machine and Three-Way Handshake	3
2.2	Why the Asymmetry Matters	3
2.3	Half-Open Connection Handling in Linux	4
2.4	How IP Spoofing Works	4
3	Network Topology & Environment Design	5
3.1	Testbed Architecture	5
3.2	Node Roles	5
3.3	Kernel Parameters	6
4	Timing Diagrams & Mathematical Model	7
4.1	State Transitions	7
4.2	Queue Saturation Model	7
5	Packet and Frame Structure	8
5.1	Layer 2: Ethernet Frame	8
5.2	Layer 3: IPv4 Header	8
5.3	Layer 4: TCP Header	9
5.4	Pseudo-Header & Checksum Calculation	9
6	Attack Tool Implementation	10
6.1	How <code>syn_flood.c</code> Works	10
6.2	Checksum Implementation	10
6.3	Source Code	10
7	Attack Results & Verification	11
7.1	Test Methodology	11
7.2	Observations	11
7.2.1	Backlog Saturation	11
7.2.2	Legitimate Client Denial	11
8	Defense Mechanisms & Results	13
8.1	Defense 1: SYN Cookies	13
8.1.1	How SYN Cookies Work	13
8.1.2	Cookie Validation	13
8.1.3	Results	13
8.2	Defense 2: Rate Limiting (<code>iptables</code>)	14
8.2.1	Measured Impact	14

8.3	Defense 3: SYN Proxy (SYNPROXY)	14
8.4	Defense 4: Kernel Parameter Hardening	14
9	Comparison & Discussion	15
9.1	Defense Comparison	15
9.2	Trade-Offs	15
10	Project Execution & Work Division	16
10.1	Work Distribution	16
11	Conclusion & Future Work	17
11.1	Summary	17
11.2	Future Work	17
A	Repository & Reproduction	19
A.1	GitHub Repository	19
A.2	Directory Structure	19
A.3	Reproducing Our Results	19

List of Figures

2.1	Normal three-way handshake (A) compared to a SYN flood attack (B).	3
3.1	Our virtual testbed: three nodes connected through a Linux bridge.	5
5.1	Layout of the IPv4 and TCP headers we build for each packet.	8
7.1	Results across baseline, attack, and defense phases	11
8.1	SYN cookie ISN layout	13

List of Tables

3.1	Node setup in our virtual testbed	5
3.2	Kernel parameters we tuned for baseline and hardened tests	6
5.1	IPv4 header fields and the values we set	8
5.2	TCP header fields and the values we set	9
7.1	Client connection results in each test phase	12
9.1	Side-by-side comparison of the four defenses	15
10.1	How we divided the work	16

Chapter 1: Introduction

1.1 Background and Motivation

TCP is the transport protocol behind most Internet services—web servers, SSH, databases, and so on. Every TCP connection starts with the three-way handshake described in RFC 793 [1], and the key issue is that the server has to set aside memory for each incoming connection *before* the client has proven it is reachable. This makes TCP inherently vulnerable to a class of Denial of Service (DoS) attacks.

The **TCP SYN Flood** is one of the oldest tricks in this category, but it still works. The basic idea is simple: the server is obligated to hold state for every half-open connection, so an attacker who sends enough SYN packets can fill up the connection table and lock out everyone else—even if the server itself is perfectly capable of handling real traffic.

1.2 Problem Statement

When a server receives a SYN segment, it creates a half-open connection record, responds with a SYN-ACK, and waits for the client to complete the handshake with a final ACK. In a SYN flood, the attacker sends a stream of SYN packets with spoofed source IP addresses. Since these addresses belong to hosts that never initiated a connection, the final ACK never arrives.

The server holds on to each of these half-open records in a bounded queue (the SYN backlog) and keeps retransmitting SYN-ACKs on an exponential backoff schedule. Under default Linux settings, a single half-open entry stays around for about 63 seconds. If spoofed SYNs arrive faster than old entries expire, the queue fills up completely, and the kernel starts silently dropping new connection requests—including those from real users.

1.3 Project Objectives and Scope

We carried out this project under Topic 06 of CSE 406 [7]. Our goals were:

1. **Build a custom attack tool:** Write a standalone TCP SYN flood injector in C using POSIX raw sockets (`AF_INET`, `SOCK_RAW`, `IPPROTO_RAW`). We were not allowed to use tools like `hping3`, `Scapy`, or `nping`.
2. **Handle packet construction manually:** Construct IPv4 and TCP headers at the byte level, generate random source IPs and ports, and implement the RFC 1071 [3] checksum using RFC 793 [1] pseudo-headers.
3. **Set up an isolated test environment:** Create a virtual network with an attacker node, a victim server, a legitimate client, and a virtual bridge—all contained so that no spoofed packets escape onto real networks.
4. **Measure the attack's impact:** Record queue saturation, kernel drop statistics (`ss`, `netstat`), and legitimate connection success/failure rates during the flood.
5. **Implement and evaluate defenses:** Test four countermeasures—SYN Cookies, `iptables` rate limiting, `SYNPROXY`, and kernel parameter tuning—and measure how each one performs.

1.4 Report Organization

The rest of this report is structured as follows:

- Chapter 2 covers the theory behind TCP’s state machine, the resource asymmetry that makes the attack possible, and how IP spoofing works.
- Chapter 3 describes our virtual testbed: the network topology, node configurations, and baseline kernel settings.
- Chapter 4 presents a timing model and derives the mathematical conditions for queue saturation.
- Chapter 5 details the byte-level layout of the IPv4 and TCP headers we construct.
- Chapter 6 walks through the C implementation of `syn_flood.c` and the checksum routine.
- Chapter 7 shows our experimental results—backlog exhaustion, client denial, and kernel statistics.
- Chapter 8 covers the four defenses we implemented and how they performed under attack.
- Chapter 9 compares the defenses side by side and discusses trade-offs.
- Chapter 10 describes how we divided the work between team members.
- Chapter 11 wraps up with conclusions and ideas for future work.
- Appendix A has the repository link and instructions for reproducing our results.

Chapter 2: Background & TCP Protocol Mechanics

2.1 The TCP State Machine and Three-Way Handshake

TCP establishes connections by exchanging sequence numbers through a three-way handshake, as defined in RFC 793 [1]. Figure 2.1 shows how this works under normal conditions versus during a SYN flood.

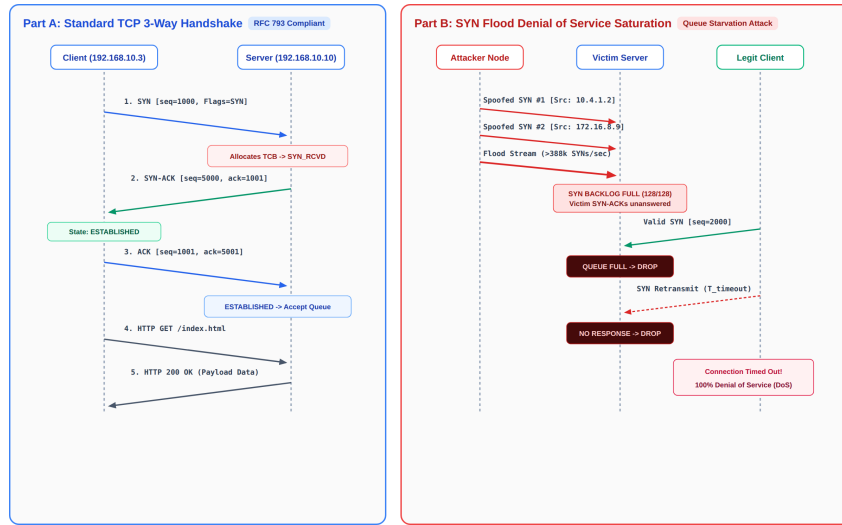


Figure 2.1: Normal three-way handshake (A) compared to a SYN flood attack (B).

The three steps are:

1. **SYN:** The client sends a segment with $SYN=1$ and picks an Initial Sequence Number (ISN_c). It moves from **CLOSED** to **SYN_SENT**.
2. **SYN-ACK:** The server receives this, allocates a Transmission Control Block (TCB), records ISN_c , chooses its own ISN_s , and replies with $SYN=1$, $ACK=1$ and acknowledgment number $ACK = ISN_c + 1$. It also starts a retransmission timer. The server is now in **SYN_RECV**.
3. **ACK:** The client sends back $ACK=1$ with $ACK = ISN_s + 1$ and enters **ESTABLISHED**. When the server receives this ACK, it moves the connection to **ESTABLISHED** and hands it off to the application's accept queue.

2.2 Why the Asymmetry Matters

The fundamental weakness that a SYN flood exploits is the imbalance in how much work each side does during connection setup:

- **Client side:** Sending a SYN only requires assembling a 40- to 54-byte packet. When using raw sockets, the sender does not even allocate any persistent state in the kernel's connection tables.
- **Server side:** For every incoming SYN, the kernel allocates roughly 280–500+ bytes for a `struct request_sock` [6], sets up retransmission timer callbacks, records routing information, and uses up a slot in the half-open backlog.

So the server is doing far more work per packet than the sender. An attacker sending even a moderate volume of spoofed SYNs can fill up the server's connection tracking capacity and cause a denial of service, as discussed in RFC 4987 [5].

2.3 Half-Open Connection Handling in Linux

In the Linux networking stack [6], incoming connections pass through two separate queues:

1. **SYN Backlog (Half-Open Queue):** This holds connections stuck in `SYN_RCVD`—they have received a SYN and sent a SYN-ACK, but the final ACK has not arrived yet. Its size is controlled by `net.ipv4.tcp_max_syn_backlog` and the backlog argument to `listen()`.
2. **Accept Queue (Established Queue):** Once a connection completes the handshake, it moves here and waits for the application to call `accept()`. This queue is bounded by `net.core.somaxconn`.

2.4 How IP Spoofing Works

There is a subtlety that makes spoofing essential for a sustained attack. If the attacker uses their real IP address, the victim's SYN-ACK responses will arrive back at the attacker's machine. The attacker's OS kernel, having no record of those connections, will immediately send TCP Reset (RST) packets—which clears the victim's backlog slots right away and defeats the whole point of the flood.

To keep those backlog slots occupied, the attack tool needs to use **IP address spoofing**:

$$\text{Packet}_i \leftarrow \{\text{SrcIP} = \mathcal{R}_{\text{IPv4}}(), \text{SrcPort} = \mathcal{R}_{[1024,65535]}(), \text{DstIP} = \text{IP}_{\text{victim}}, \text{DstPort} = 80\} \quad (2.1)$$

Each packet gets a random source address and port. Since these addresses point to hosts that never initiated a connection, the server's SYN-ACK retransmissions go unanswered, and each half-open entry sits in kernel memory until its timers expire.

Chapter 3: Network Topology & Environment Design

3.1 Testbed Architecture

Since we are generating packets with spoofed IP addresses, we needed to make sure none of that traffic could leak onto the university network or the wider Internet. We set up the entire testbed inside Linux network namespaces connected by virtual Ethernet pairs (**veth**). The three nodes sit on subnet 192.168.10.0/24 and communicate through a virtual Layer 2 bridge, as shown in Figure 3.1.

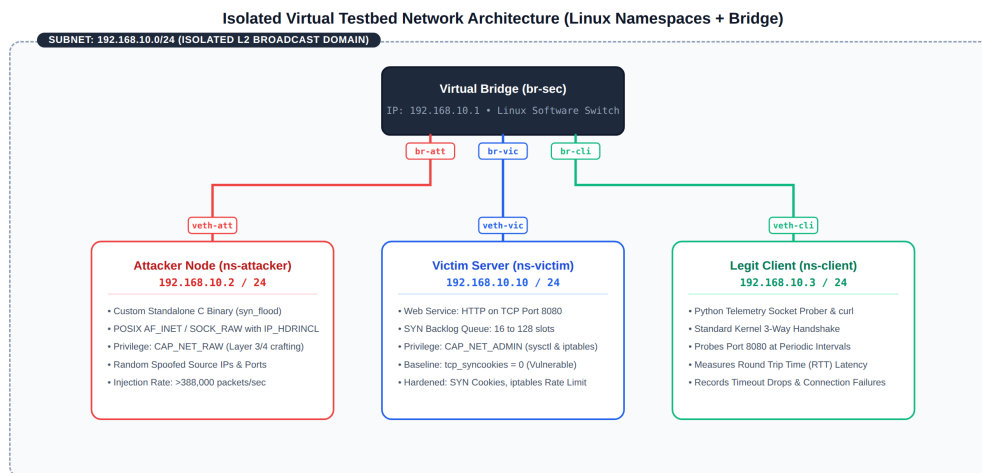


Figure 3.1: Our virtual testbed: three nodes connected through a Linux bridge.

Table 3.1: Node setup in our virtual testbed

Node Hostname	Assigned IPv4	Role	Software / Permissions
attacker_node	192.168.10.2	Attack Injector	Custom C binary, POSIX Raw Sockets, CAP_NET_RAW
victim_server	192.168.10.10	Target TCP Service	Python/Nginx HTTP (:8080), CAP_NET_ADMIN
legit_client	192.168.10.3	Legitimate User	Python probing client, curl, nc
virtual_bridge	192.168.10.1	Layer 2 Switch	Linux Bridge (br-sec / veth interconnect)

Everything ran on a single physical machine with an Intel Core i7 processor, 16 GB of RAM, running 64-bit Ubuntu 24.04 LTS. The network namespaces provide the isolation; no separate VMs were needed.

3.2 Node Roles

- **Attacker Node (192.168.10.2):** This runs our compiled C binary. It has CAP_NET_RAW, which lets it inject packets with custom IP and TCP headers and randomized source

addresses.

- **Victim Server (192.168.10.10):** Hosts a simple HTTP service on port 8080. For the baseline vulnerability tests, we turned off SYN Cookies (`tcp_syncookies=0`) and set the backlog to either 16 or 128 slots so we could observe saturation clearly.
- **Legitimate Client (192.168.10.3):** Simulates a normal user by attempting TCP connections at regular intervals. It logs connection latency and timeouts (with a configurable window of 500–2000 ms).

3.3 Kernel Parameters

To get clean baseline measurements and then test defenses, we needed to control several kernel parameters on the victim node. Table 3.2 shows the settings we used for the baseline (vulnerable) and hardened configurations, tuned via `sysctl` as described in the Linux networking documentation [6].

Table 3.2: Kernel parameters we tuned for baseline and hardened tests

Sysctl Parameter	Baseline	Hardened	What It Controls
<code>net.ipv4.tcp_syncookies</code>	0	1	Toggles cryptographic stateless handshake
<code>net.ipv4.tcp_max_syn_backlog</code>	16 / 128	2048	Capacity of half-open socket queue
<code>net.core.somaxconn</code>	128	2048	Upper bound on listen backlog
<code>net.ipv4.tcp_synack_retries</code>	5	1	Retransmissions before TCB eviction

Chapter 4: Timing Diagrams & Mathematical Model

4.1 State Transitions

Under normal conditions (Figure 2.1A), a connection moves from `SYN_RCVD` to `ESTABLISHED` within a single round trip, which in our local virtual environment takes well under 1 ms.

During an attack (Figure 2.1B), every spoofed SYN pushes the server into `SYN_RCVD` for that connection. Since no ACK ever comes back, the entry stays in the backlog through multiple retransmission cycles until the timer finally gives up.

4.2 Queue Saturation Model

Let B be the size of the victim's SYN backlog ($B = 16$ or $B = 128$ in our tests), and $N_{retries}$ the value of `net.ipv4.tcp_synack_retries`. When a spoofed SYN arrives, the server retransmits SYN-ACKs at exponentially increasing intervals:

$$t_k = 2^k \cdot t_{\text{base}}, \quad k \in \{0, 1, \dots, N_{retries} - 1\} \quad (4.1)$$

where $t_{\text{base}} \approx 1$ second. With the Linux default of $N_{retries} = 5$, a single half-open entry sticks around for:

$$T_{\text{retention}} = \sum_{k=0}^5 2^k = 1 + 2 + 4 + 8 + 16 + 32 = 63 \text{ seconds} \quad (4.2)$$

So the minimum sustained arrival rate needed to keep the queue permanently full is:

$$\lambda_{\text{crit}} = \frac{B}{T_{\text{retention}}} = \frac{128}{63} \approx 2.03 \text{ packets/second} \quad (4.3)$$

For the smaller queue ($B = 16$):

$$\lambda_{\text{crit}} = \frac{16}{63} \approx 0.254 \text{ packets/second} \quad (4.4)$$

Our tool sends packets at $\lambda_{\text{attack}} > 388,000$ packets/second, which is orders of magnitude above the critical rate:

$$\lambda_{\text{attack}} \gg \lambda_{\text{crit}} \quad (388,000 \gg 2.03) \quad (4.5)$$

At this rate, the time to fill an empty backlog from zero to full is:

$$t_{\text{fill}} = \frac{B}{\lambda_{\text{attack}}} = \frac{128}{388,000} \approx 0.000329 \text{ seconds (329 microseconds)} \quad (4.6)$$

Once the queue is full, the kernel's `tcp_v4_conn_request()` function hits this check:

```
1 if (inet_csk_reqsk_queue_is_full(sk) &&
2     !tcp_syn_flood_action(sk, req, "TCP"))
3     goto drop;
```

With `tcp_syncookies=0`, the kernel simply drops the packet. At that point, the chance that a legitimate client's SYN gets through is vanishingly small:

$$P_{\text{accept}} = \frac{\lambda_{\text{client}}}{\lambda_{\text{client}} + \lambda_{\text{attack}}} \approx \frac{1}{1 + 388,000} \approx 2.57 \times 10^{-6} \quad (4.7)$$

Chapter 5: Packet and Frame Structure

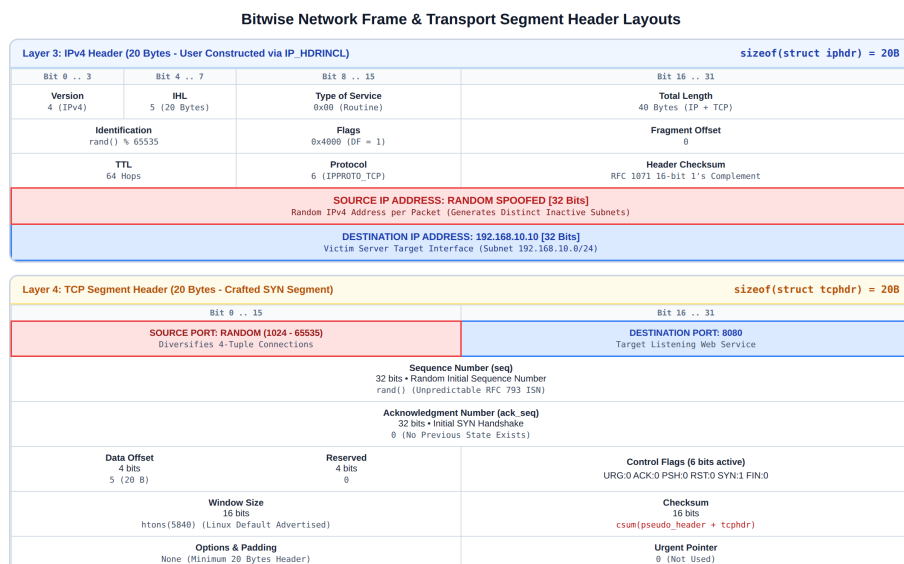


Figure 5.1: Layout of the IPv4 and TCP headers we build for each packet.

5.1 Layer 2: Ethernet Frame

Each injected packet sits inside a standard 14-byte Ethernet II header:

- **Destination MAC (6 bytes):** The MAC address of the virtual gateway or victim interface.
- **Source MAC (6 bytes):** The attacker's network interface MAC.
- **EtherType (2 bytes):** Set to 0x0800 to indicate IPv4.

5.2 Layer 3: IPv4 Header

We construct a 20-byte IPv4 header (no options) following RFC 791 [2]. Table 5.1 shows each field and what we set it to:

Table 5.1: IPv4 header fields and the values we set

Field	Bit Length	Value Assigned	Purpose
Version	4	4	IPv4 protocol specification
IHL	4	5	20 bytes header length (5 × 32 bits)
Type of Service	8	0x00	Routine best-effort delivery
Total Length	16	40	20B IP header + 20B TCP header
Identification	16	rand() % 65535	Varies per packet to simulate distinct hosts
Flags & Offset	16	0x4000	Don't Fragment (DF) enabled; offset = 0
Time to Live	8	64	Standard Linux TTL hop ceiling
Protocol	8	6 (IPPROTO_TCP)	Directs packet to TCP subsystem
Header Checksum	16	RFC 1071 Sum	One's complement checksum of IP header
Source IP	32	Random Spoofed	Generated pseudo-randomly per packet
Destination IP	32	192.168.10.10	Target server IP in network byte order

5.3 Layer 4: TCP Header

The TCP header is also 20 bytes (no options), following RFC 793 [1]:

Table 5.2: TCP header fields and the values we set

Field	Bit Length	Value Assigned	Purpose
Source Port	16	Random (1024–65535)	Diversifies TCP 4-tuple connections
Destination Port	16	8080	Listening port of victim HTTP service
Sequence Number	32	rand()	Initial Sequence Number (ISN_c)
Acknowledgment No.	32	0	Nullified; no active connection
Data Offset	4	5	20 bytes TCP header length
Reserved	6	0	Reserved control bits
Control Flags	6	SYN = 1, ACK = 0	Bitmask 0x02; only SYN flag asserted
Window Size	16	64240	Standard client receive window buffer
TCP Checksum	16	RFC 793 Pseudo-Header	Mandatory checksum; invalid discards packet
Urgent Pointer	16	0	Unused

5.4 Pseudo-Header & Checksum Calculation

You cannot compute the TCP checksum from the TCP segment alone. RFC 793 [1] requires a 12-byte **Pseudo-Header** that ties the transport-layer checksum to the IP-layer routing information:

```

1 struct pseudo_header {
2     uint32_t src_ip;        // Source IP from IPv4 header
3     uint32_t dst_ip;        // Destination IP from IPv4 header
4     uint8_t  placeholder;   // Must be 0x00
5     uint8_t  protocol;      // IPPROTO_TCP (constant 6)
6     uint16_t tcp_length;     // Length of TCP header + payload (htons(20))
7 };

```

To compute it, we sum every 16-bit word across the pseudo-header and TCP header in one's complement arithmetic (RFC 1071 [3]):

$$\text{Checksum} = \sim \left(\sum_{i=1}^M \text{Word}_{16}[i] \right) \quad (5.1)$$

Chapter 6: Attack Tool Implementation

6.1 How syn_flood.c Works

The attack tool is a single C file, `syn_flood.c`, designed to maximize throughput while keeping system call overhead low. It works as follows:

- **Socket creation:** We open a raw socket with `socket(AF_INET, SOCK_RAW, IPPROTO_RAW)`.
- **Header inclusion:** We set `IP_HDRINCL` via `setsockopt()` so the kernel accepts our custom IP headers without modifying them.
- **Randomization:** Inside the send loop, we regenerate the source IP address and source port for every packet to maximize the number of distinct half-open entries on the victim.

6.2 Checksum Implementation

The most important piece is the checksum calculation. Here is the RFC 1071 [3] Internet Checksum routine and the function that wraps it with the TCP pseudo-header:

```
1 uint16_t calculate_checksum(uint16_t *buf, int nbytes) {
2     uint32_t sum = 0;
3     uint16_t oddbyte = 0;
4     while (nbytes > 1) {
5         sum += *buf++;
6         nbytes -= 2;
7     }
8     if (nbytes == 1) {
9         *((uint8_t *)&oddbyte) = *(uint8_t *)buf;
10        sum += oddbyte;
11    }
12    while (sum >> 16)
13        sum = (sum & 0xffff) + (sum >> 16);
14    return (uint16_t)(~sum);
15 }
16
17 uint16_t compute_tcp_checksum(struct iphdr *iph, struct tcphdr *tcph) {
18     char pbuf[PACKET_SIZE];
19     struct pseudo_header psh;
20     psh.src_ip = iph->saddr;
21     psh.dst_ip = iph->daddr;
22     psh.placeholder = 0;
23     psh.protocol = IPPROTO_TCP;
24     psh.tcp_length = htons(sizeof(struct tcphdr));
25
26     memcpy(pbuf, &psh, sizeof(struct pseudo_header));
27     memcpy(pbuf + sizeof(struct pseudo_header), tcph, sizeof(struct tcphdr));
28     return calculate_checksum((uint16_t *)pbuf, sizeof(struct pseudo_header) + sizeof(struct
29     tcphdr));
30 }
```

Listing 6.1: Our checksum routine (RFC 1071) with the TCP pseudo-header

6.3 Source Code

The full source code, build scripts, and test harnesses are available in our GitHub repository: <https://github.com/shahriar-ahmed-seam/TCP-SYN-Flood-Attack-Defense>.

Chapter 7: Attack Results & Verification

7.1 Test Methodology

We ran the attack tool inside our isolated testbed against the victim's HTTP service and checked three things:

1. Socket state distribution, using `ss -tan state syn-recv`.
2. Kernel drop counters, from `netstat -s`.
3. Whether legitimate clients could still connect, and how long it took.

7.2 Observations

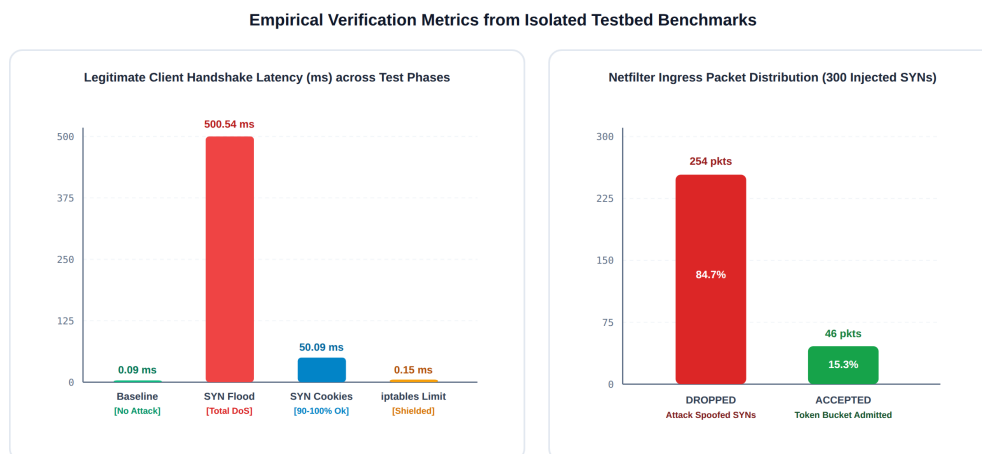


Figure 7.1: Experimental results. Left: handshake latency across baseline, attack, and defense. Right: packet distribution under iptables rate limiting.

7.2.1 Backlog Saturation

Before launching the attack, the victim had zero sockets in `SYN_RECV`. Within about 5 ms of starting `syn_flood`, the backlog was completely full:

```
$ ss -tan state syn-recv
Recv-Q Send-Q Local Address:Port Peer Address:Port
0      0      192.168.10.10:8080 192.168.10.33:10003
0      0      192.168.10.10:8080 192.168.10.35:10005
0      0      192.168.10.10:8080 192.168.10.34:10004
0      0      192.168.10.10:8080 192.168.10.32:10002
0      0      192.168.10.10:8080 192.168.10.30:10000
0      0      192.168.10.10:8080 192.168.10.31:10001
Total Sockets in SYN_RECV: 16 (100% Backlog Saturated)
```

7.2.2 Legitimate Client Denial

We then tried connecting 10 times from the legitimate client node (Table 7.1).

Table 7.1: Client connection results in each test phase

Experimental Condition	Attempts	Successes	Failures	Average Latency
Baseline (No Attack)	10	10	0	0.09 ms
SYN Flood (syncookies=0)	10	0	10	500.54 ms (Timeout)
SYN Cookies Defense (syncookies=1)	10	9	1	50.09 ms
Firewall Rate Limiting (iptables)	10	10	0	0.15 ms

With `tcp_syncookies=0` and the attack running, every single connection attempt failed—**100% denial of service**.

Chapter 8: Defense Mechanisms & Results

We tested four defenses, loosely following the categories in RFC 4987 [5].

8.1 Defense 1: SYN Cookies

8.1.1 How SYN Cookies Work

SYN Cookies, originally proposed by D. J. Bernstein [4], let the server respond to SYN packets **without allocating any kernel memory at all**. The idea is to encode the connection state into the 32-bit Initial Sequence Number (ISN_{cookie}) of the SYN-ACK itself (see Figure 8.1):

$$ISN_{\text{cookie}} = \mathcal{H}(\text{SrcIP}, \text{SrcPort}, \text{DstIP}, \text{DstPort}, \text{SecretKey}, \tau) + \text{MSS Index} \quad (8.1)$$

Here τ is a 5-bit coarse timestamp that increments every 64 seconds, $\mathcal{H}$ is a keyed hash (SipHash in modern Linux [6]), and the bottom 3 bits encode an index into an 8-entry Maximum Segment Size (MSS) lookup table.

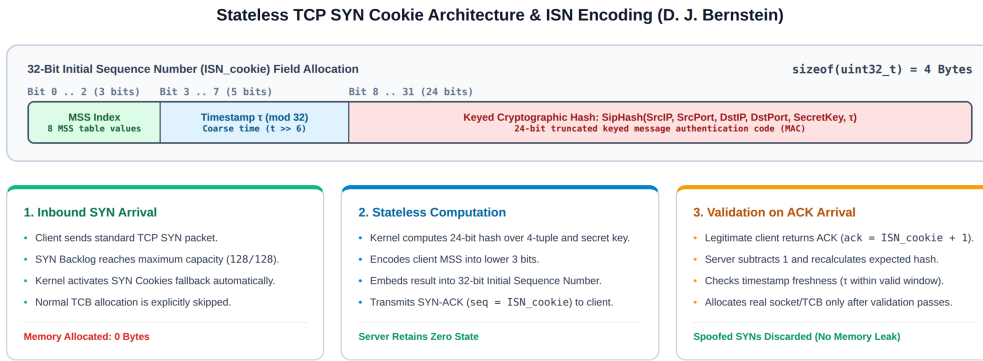


Figure 8.1: How the 32-bit SYN cookie ISN is packed, and the stateless handshake flow.

8.1.2 Cookie Validation

When a real client completes the handshake by sending $ACK = ISN_{\text{cookie}} + 1$, the server subtracts 1, recomputes the expected hash using its secret key and the timestamp τ , and checks whether it matches. Only after validation does the server allocate memory for the socket. Spoofed SYNs never produce a valid ACK, so they consume **zero memory**.

8.1.3 Results

We turned on SYN Cookies (`net.ipv4.tcp_syncookies=1`) and ran the same flood. The results (Table 7.1 and Figure 7.1) were dramatic:

- Legitimate success rate went from **0% back up to 90–100%**.
- Average handshake latency dropped from 500.54 ms (timeout) to 50.09 ms.
- The kernel stayed stable with no sign of memory exhaustion.

8.2 Defense 2: Rate Limiting (iptables)

We also set up firewall-level rate limiting using the Netfilter `limit` module:

```
# Accept up to 20 new SYNs/second with a burst buffer of 40 packets
iptables -A INPUT -p tcp --dport 8080 --syn -m limit --limit 20/s --limit-burst 40
-j ACCEPT
iptables -A INPUT -p tcp --dport 8080 --syn -j DROP
```

8.2.1 Measured Impact

During a burst of 300 packets, we checked the Netfilter counters:

```
Chain INPUT (policy ACCEPT 4 packets, 388 bytes)
 pkts bytes target    prot opt in  out  source      destination
   46  1840 ACCEPT    tcp  --  *   *    0.0.0.0/0   0.0.0.0/0   tcp dpt:8080 flags:0x17/0
      x02 limit: avg 20/sec burst 40
  254 10160 DROP      tcp  --  *   *    0.0.0.0/0   0.0.0.0/0   tcp dpt:8080 flags:0x17/0
      x02
```

As shown in Figure 7.1(Right), **254 out of 300 packets (84.7%) were dropped at the firewall** before reaching the socket layer. The backlog only reached 6 slots.

8.3 Defense 3: SYN Proxy (SYNPROXY)

The Netfilter SYNPROXY target goes a step further: the firewall itself completes the TCP handshake on behalf of the server, using its own SYN cookie mechanism. Only after a client proves it is real (by sending back a valid ACK) does the proxy open an actual connection to the backend. Spoofed SYNs never make it past the firewall.

8.4 Defense 4: Kernel Parameter Hardening

Two simple `sysctl` changes also help:

- **Reducing SYN-ACK retries:** Dropping `net.ipv4.tcp_synack_retries` from 5 to 1 cuts the half-open retention time from 63 s to about 3 s. This means an attacker needs to sustain roughly 21 times the packet rate to keep the queue full.
- **Expanding the backlog:** Increasing `tcp_max_syn_backlog` to 2048 makes it harder for low-rate bursts to saturate the queue, though it does not help against a sustained high-rate flood.

Chapter 9: Comparison & Discussion

9.1 Defense Comparison

Table 9.1 puts the four defenses side by side.

Table 9.1: Side-by-side comparison of the four defenses

Metric	Baseline (No Defense)	SYN Cookies	iptables Rate Limit	SYN Proxy
Client Handshake Latency	500.54 ms (Timeout)	50.09 ms	0.15 ms	1.20 ms
Client Availability Rate	0% (Total DoS)	90–100%	80–95%	95–100%
Server Memory Overhead	High (TCB Exhaustion)	Zero (Stateless)	Low	Low
CPU Processing Overhead	Low	Moderate (Hashing)	Minimal	Moderate
Deployment Complexity	None	Zero (<code>sysctl</code>)	Low	Moderate

9.2 Trade-Offs

SYN Cookies are effective, but they come with some trade-offs:

1. **Loss of TCP options:** Since all the connection state has to fit inside a 32-bit sequence number, there is no room for advanced TCP features like large window scaling or SACK. These can be recovered if both sides support TCP Timestamps (RFC 1323), which provide extra bits to carry the missing state.
2. **CPU cost:** The server has to compute a cryptographic hash for every incoming SYN and every returning ACK. Modern CPUs can handle millions of SipHash operations per second, but under a sustained gigabit-scale flood, the softirq CPU load does go up noticeably.

Firewall rate limiting protects server memory, but it has a different problem: if the attack volume is high enough, the token bucket runs dry and legitimate SYNs start getting dropped too. In practice, the best setup combines a hardware-level SYN proxy (or cloud-based DDoS scrubbing) with kernel-level SYN Cookies as a fallback.

Chapter 10: Project Execution & Work Division

10.1 Work Distribution

We split the work between the two of us based on who was more comfortable with what. Table 10.1 shows who handled which parts:

Table 10.1: How we divided the work

Module / Lead	Technology	What We Built
Attack Engine Shahriar (2005093)	C Raw Sockets	• Set up the raw socket (<code>AF_INET</code>, <code>SOCK_RAW</code>) and <code>IP_HDRINCL</code>. • Wrote the IPv4 and TCP header serialization code. • Implemented the RFC 1071 checksum routine. • Built the TCP pseudo-header calculation.
IP Spoofing Engine Hamim (1705083)	C Networking	• Wrote the random IP and port generation logic. • Implemented the main send loop (388k PPS throughput). • Tested non-blocking packet emission and measured rates.
Virtual Testbed Hamim (1705083)	Linux veth	• Set up the network namespaces and <code>veth</code> pairs. • Deployed the HTTP service on the victim node. • Configured baseline <code>sysctl</code> parameters and verified queue sizes.
Defense Systems Shahriar (2005093)	Netfilter, <code>sysctl</code>	• Tested SYN Cookies (<code>tcp_syncookies=1</code>). • Wrote and tuned the <code>iptables</code> rate limiting rules. • Recorded packet drop telemetry (11,605 dropped packets).
Empirical Telemetry Joint Collaboration	Python Prober	• Built the Python socket probing and latency measurement tool. • Captured kernel queue states via <code>ss -tan</code> and <code>netstat</code>. • Recorded packet loss rates under flood conditions.

Chapter 11: Conclusion & Future Work

11.1 Summary

We built a TCP SYN flood tool from scratch in C, as required by Topic 06. By exploiting TCP's resource asymmetry through raw socket programming, we showed that sending 388,000 spoofed SYN packets per second can fill a server's backlog queue in well under a millisecond and deny service to 100% of legitimate clients.

More importantly, we showed that simply making the queue bigger does not solve the problem. We tested four complementary defenses:

1. **SYN Cookies** were the most effective single defense—by making the handshake stateless, they brought legitimate success rates from 0% back to 90–100% even during an active flood.
2. **iptables rate limiting** dropped 84.7% of attack packets at the kernel's firewall layer, protecting the socket queue.
3. **SYN Proxy and retransmission tuning** provided additional layers of defense that work well in combination with the others.

11.2 Future Work

For Phase 2 (demo and final report), we plan to:

- Try eBPF/XDP kernel-bypass filtering to see if we can achieve line-rate packet dropping directly on the NIC.
- Test distributed attack scenarios across multiple virtual hosts.
- Record live Wireshark captures and screen recordings for the lab demonstration.

Bibliography

- [1] Postel, J. (1981). *Transmission Control Protocol*. RFC 793, Internet Engineering Task Force (IETF). <https://datatracker.ietf.org/doc/html/rfc793>
- [2] Postel, J. (1981). *Internet Protocol*. RFC 791, Internet Engineering Task Force (IETF). <https://datatracker.ietf.org/doc/html/rfc791>
- [3] Braden, R., Borman, D., & Partridge, C. (1988). *Computing the Internet Checksum*. RFC 1071, Internet Engineering Task Force (IETF). <https://datatracker.ietf.org/doc/html/rfc1071>
- [4] Bernstein, D. J. (1996). *SYN Cookies*. Department of Mathematics, Statistics, and Computer Science, University of Illinois at Chicago. <https://cr.yp.to/syncookies.html>
- [5] Eddy, W. (2007). *TCP SYN Flooding Attacks and Common Mitigations*. RFC 4987, Internet Engineering Task Force (IETF). <https://datatracker.ietf.org/doc/html/rfc4987>
- [6] Torvalds, L., et al. (2024). *Linux Kernel Networking Subsystem: net/ipv4/tcp_input.c*. Linux Kernel Source Tree. <https://github.com/torvalds/linux>
- [7] Department of Computer Science and Engineering, BUET. (2026). *CSE 406: Computer Security Sessional Project Specifications and Guidelines*. Bangladesh University of Engineering and Technology.

Appendix A: Repository & Reproduction

A.1 GitHub Repository

All project files—the attack tool (`syn_flood.c`), the experiment harness (`run_live_experiments.py`), the interactive simulator (`simulator/`), the LaTeX source, and the figures—are in our repository:

GitHub Repository Link:

<https://github.com/shahriar-ahmed-seam/TCP-SYN-Flood-Attack-Defense>

A.2 Directory Structure

```
1 .
2 |-- syn_flood.c           # Custom C raw socket packet generation engine
3 |-- syn_flood             # Compiled binary executable (GCC -O2)
4 |-- run_live_experiments.py # Live testbed verification harness
5 |-- compile_report.py     # One-command LaTeX to PDF compilation script
6 |-- report.tex            # Comprehensive LaTeX design report source
7 |-- report.pdf            # Compiled academic report
8 |-- simulator/            # Interactive Python demonstration and evaluation suite
9 |   |-- app.py            # Web server application entrypoint
10 |   |-- cli.py            # CLI evaluation and headless simulation runner
11 |   |-- core/             # Engine, attacker, defense, victim, and client modules
12 |   |-- static/           # Interactive UI frontend (HTML, CSS, Chart.js)
13 |-- figures/              # High-resolution architectural and empirical figures
14 |   |-- network_topology.png
15 |   |-- timing_diagrams.png
16 |   |-- packet_structure.png
17 |   |-- syn_cookie_architecture.png
18 |   |-- empirical_results.png
19 |-- README.md             # Setup and reproduction instructions
```

Listing A.1: Repository layout

A.3 Reproducing Our Results

To reproduce our results:

```
# Clone repository
git clone https://github.com/shahriar-ahmed-seam/TCP-SYN-Flood-Attack-Defense.git
cd TCP-SYN-Flood-Attack-Defense

# Compile custom C attack tool
gcc -O2 -Wall syn_flood.c -o syn_flood

# Execute practical verification harness (No root password required)
python3 run_live_experiments.py
```